ALBUKHARY INTERNATIONAL UNIVERSITY

DU014 (K)

CRYPTOGRAPHY ESSENTIALS

LITERATURE REVIEW

AES-Based Secure File Encryption and Decryption Tool

Student Name	_____Maralbyek Tilyek_____
Student ID	_____AIU24102280_____
Course / Course Code	Cryptography Essentials CCC2243
Related Deliverable	Project Proposal — AES-Based Secure File Encryption and Decryption Tool
Year / Semester	Year 2, Semester 3, 2025-2026

Table of Content

1. Introduction

Unauthorized access of a computer, removable drive, or shared storage site by another person can result in the copying, viewing, or editing of information stored on digital files. To solve this issue, file encryption is used to convert the readable plaintext into a cryptographic ciphertext with a secret key and an algorithm. If the key is not supplied, the message should be impracticable to decipher.
This literature review will discuss the concepts required to build the proposed AES-Based Secure File Encryption and Decryption Tool. It covers symmetric cryptography, Advanced Encryption Standard (AES), authenticated encryption using AES-GCM, password-based key derivation, salts, nonces, file-format design and secure implementation. The sections are written to explain a particular design decision that is continued to the project proposal, and the review is completed by a direct correlation of the decisions with the methodology and evaluation criteria of the project proposal.

2. Symmetric-Key Cryptography

Symmetric-key cryptography uses the same secret key (or material) used for encryption and decryption. The sender and receiver should therefore have a secure means of communicating and exchanging the key, or generating the key, and a secure way to protect it. Symmetric algorithms are typically quicker than public-key algorithms, making them suitable for processing large data sets like documents, images, and videos.

The security of a symmetric system is based on two factors: the algorithm and the key's secrecy. If the algorithm is of good strength, but the key is compromised, repeated inappropriately, or is poorly generated, then the data is not protected. The user password will not be directly used for the proposed tool as an AES key. Rather, a key derivation function will map the password to a fixed length cryptographical key, and a random salt will add to the difficulty of password guessing attacks. This decision is directly reflected in Objective 3 and encryption workflow (Section 6.1) of the project proposal.

3. The Advanced Encryption Standard (AES)

AES is one of the block ciphers that has been standardised by the United States National Institute of Standards and Technology (NIST) in FIPS PUB 197. It runs on 128-bit blocks and keys of 128, 192 and 256 bits. AES is a replacement for the previously used Data Encryption Standard, which had a relatively weak 56-bit key that was the subject of exhaustive key-search attacks as computer capability grew.

AES is a primitive that encrypts a fixed size block at a time. AES can only be implemented in an operation mode to encrypt an entire file. The mode dictates how several blocks are going to be processed and if other security properties are going to be offered, such as authentication. So it is not enough to choose AES: one has to choose a secure mode, generate the necessary random

values in the right way and manage keys and errors in a secure fashion. That's the reason the proposal includes not only AES but the full construction AES-256-GCM.

4. AES-128 and AES-256

AES is the 128-bit block size encryption, AES-128 and AES-256 have the same block size but different key lengths. AES-256 is more resistant to exhaustive search and has more key space. AES-128 is already suitable for many uses and AES-256 is a good choice in order to demonstrate good protection and future-oriented security for a student project. The variant that is used in the project proposal (Section 7, Tools and Technologies) is AES-256.

That's not to say that the larger key is a free pass for bad implementation. A weak password or a predictable nonce, reused nonce or incorrect file handling can compromise either AES-128 or AES-256. Therefore, the proposed tool will focus on a well reviewed cryptographic library, rather than implementing AES mathematical operations by hand, specifically the Python cryptography library as outlined in the proposal.

5. Authenticated Encryption and AES-GCM

While the content of the file is hidden and cannot be seen by other users, this does not mean that the encrypted file hasn't been altered. The attacker could make a mistake and corrupt the ciphertext, inject a different file, or modify it in some way. A secure file-encryption tool should thus be able to ensure both confidentiality and integrity.

Galois/Counter Mode (GCM) is a mode of operation for block ciphers that is authenticated-encryption. AES-GCM encrypts the plaintext and generates a 128 bit (16-byte) authentication tag. On decryption the tag is verified prior to acceptance of the plaintext. If the authentication is incorrect (because the password is wrong, or the ciphertext, nonce, associated data or tag is changed), the application should reject the result, not output corrupted or unauthenticated plaintext. This is the exact behavior that is specified in the decryption workflow (Step 3 in Section 6.2) of this proposal and is mirrored in the "Security" row of its Testing and Evaluation Criteria table.

One of the important requirements of AES-GCM is that the nonce should never be used twice with the same key. The application should be able to produce a new, truly random 96-bit (12-byte) nonce for each operation of encryption — just the size that the proposal's methodology requires. The nonce doesn't have to be private.The proposed system will rely on a cryptographically secure random nonce, and will consider reusing a nonce to be a serious implementation mistake, as detailed in the threat model of the proposed system in Section 6.3.

6. Passwords, Keys, and PBKDF2

Usually, only passwords are remembered by users, not the keys for the random binary encryption. Often passwords are not as long or random as keys and can be guessed using a dictionary or brute force. To make the guess harder, a password-based key derivation function (KDF) is used to repeatedly apply a pseudorandom function to generate a key of the desired length.

The password-based key derivation function (PBKDF2) is a cryptographic technique defined in RFC 8018 that is used to protect passwords. It is a combination of password, salt, a pseudorandom function like HMAC-SHA-256 and an iteration count. The salt is public random data, in the proposed design, it's a cryptographically random 16 bytes that makes sure that the same password generates different keys for encryption in different instances. It also makes it more difficult to use a pre-computed password table efficiently on multiple files by an attacker.

The iteration count determines the cost of the computation to get a key. Based on OWASP OWSAP passwords storage guidelines adapted for key derivation, OWASP OWASP-OWAP proposed application will use at least 480,000 iterations of the PBKDF2-HMAC-SHA256 key derivation function. This value should also be checked on the target development machine to ensure that password guessing is costly, but that normal encryption and decryption are still reasonably fast. The number of iterations is stored in the encrypted file header so that the decryption can repeat the process and create the same iterations. PBKDF2 isn't a way to make a weak password strong, it merely makes a large-scale guess more expensive. This application should, then, promote — but not force — the use of long, complex passwords.

7. Salt, Nonce, Authentication Tag, and File Format

Several values have to be preserved in an encrypted file in order to decrypt it. It is necessary to use the same key to derive the same key for the salt. AES-GCM must reproduce the encryption context, and requires the nonce. The authentication tag is needed to ensure that the ciphertext has not been altered. These values are not guarded secrets, but they must be stored correctly and safely retrieved.

A practical output format might start with a small header with the version number, followed by the salt, nonce, authentication tag and the ciphertext, as described in the project proposal (Section 6.1, Step 5) in the binary format. The version field can be used to change the format of the application in the future without mixing old and new files. It is possible to store the original name (not sure if this is useful, but it's stored).It is possible to store the original filename, but the application should try to make it not visible if it is not necessary for the sensitive data. The output should be in a unique extension like .enc like the example in the proposal: document.pdf.enc, which is to distinguish it from regular files.

The application should decrypt and store in a temporary file (and not replace or create the destination until after successful AES-GCM authentication). This ensures that an invalid password or

a compromised ciphertext won't create a misleading partial output, and directly addresses a threat that was described in the proposal model (Section 6.3) of interrupted or incomplete writes.

8. Secure Software Implementation

Because it is hard to implement cryptographic software safely, small errors can destroy otherwise good math. The proposed project should rely on a well-established library, like Python's cryptography library, instead of coding AES, GCM, or PBKDF2. Library APIs should be used as specified and any exceptions should be managed without revealing passwords, keys or sensitive plain text in messages or logs.

Passwords should be collected, without showing any character, and should not be stored after the operation. Only store keys and plaintext for as long as they are needed. General error messages, including "decryption failed" should be displayed at interface, rather than revealing that the password is not quite correct or the specific portion of the encrypted file is not valid. These practices will minimize information leakage while supporting the clear messaging requirement of the project proposal Objective 5, which is non-technical in nature.

9. Related Applications and Security Considerations

Existing file-encryption applications have shown that people require more than just an encryption button. They require to select files clearly, to have status messages that are easy to understand, to be protected from unwanted file overwriting, and to have the ability to recover files in the event of an accident. There are some applications that combine encryption and operating-system authentication, cloud sync, key management, and password managers. Those are not the features that would be found in a small academic tool, nor is cloud integration, multi-user access control, or enterprise key management included in the scope of the proposal; but the proposal itself can show the basic cryptographic flow appropriately.

Main threats for this project are the use of weak password, lost password, insecure storage of keys, nonce re-use, deletion of the original file, and lack of proper verification of the authenticity before releasing any decrypted data. Each of these risks aligns directly with the three risks that were identified in the threat model of the proposal (Section 6.3): Weak/Reused Passwords, Nonce Reuse, Interrupted Writes. Encryption will NOT retrieve a forgotten password and will NOT protect a file once it has been cracked. These limitations should be explicitly written in the project documentation as stated in the Out of Scope section of the proposal "password recovery".

10. Comparison of Design Options

Option	Strengths	Limitations / Decision
AES-GCM	Provides encryption and tamper detection in one standard mode.	Requires strict nonce uniqueness. Selected for this project.

AES-CBC with separate MAC	Well-known confidentiality mode; can be combined with HMAC.	More complex to implement correctly because encryption and authentication must be combined safely. Not selected.
AES-128	Strong security and slightly lower computational cost.	Smaller security margin than AES-256. Not selected for the main prototype.
AES-256	Large key space and strong long-term security margin.	Requires correct password-based key derivation; selected for the project.
Direct password use	Simple to explain and code.	Unsafe because human passwords are not uniformly random keys. Rejected.
PBKDF2-derived key (480,000 iterations)	Standardized, available in trusted libraries, and slows guessing.	Still depends on password quality and a suitable iteration count. Selected.

11. Relevance to the Proposed Project

Proposed architecture is supported by the reviewed research.The application backend will be provided by Python's built-in HTTP server, while the graphical interface will be provided by a browser-based HTML, CSS, and JavaScript frontend. The cryptography package will provide tested implementations of AES-GCM and PBKDF2 as listed in the Tools and Technologies table in the proposal. For every encryption operation, the application will generate a new 16-byte salt and 12-byte nonce; it will use 480,000 iterations of PBKDF2-HMAC-SHA256 on the user's password to generate a 256-bit key; it will store the non-secret parameters with the encrypted data; and it will verify the authenticity tag before creating a decrypted file.

The literature also sets the criteria for the project, which are directly translated into the five categories listed in the Testing and Evaluation Criteria table in the project proposal. Correctness is verified by using successfully encrypted and decrypted files and comparing the resulting hashes with the ones computed from the original content using the same hash function, SHA-256; Security is verified by rejecting the invalid password attempts and also by the rejection of the encrypted file that has been modified; Robustness is verified by preserving the contents of the binary file and by the correct behavior with empty files and very large files; Usability is verified by the protection against accidental overwriting and by the clear error messages; and Performance is measured against the proposal's claim of < 5s encryption/decryption time on a 100 MB file and is done using

the pytest test suite, specified in the proposal's tools. These tests will involve both functional and security property tests for the application.

12. Conclusion

AES is a suitable foundation for a file-encryption program for the local system, since it is widely supported, efficient and standardized. Secure file encryption isn't as simple as picking a good algorithm: it's a combination of an authenticated mode, unique nonces, password-based key derivation, random salts, secure file-format design, and safe error handling. AES-256-GCM with a key derived from a password using the PBKDF2-HMAC-SHA256 function with 480,000 iterations would therefore be a good design for the proposed project, assuming that a trusted cryptographic library is used and that the limitations of password-based protection, as already identified in the project proposal's scope and threat model, are explicitly documented in the final report.

References

Dworkin, M. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38D

Kaliski, B. (2017). *PKCS #5: Password-based cryptography specification version 2.1* (RFC 8018). Internet Engineering Task Force. https://doi.org/10.17487/RFC8018

National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)* (FIPS PUB 197). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.197

National Institute of Standards and Technology. (2010). *Recommendation for password-based key derivation: Part 1, storage applications* (NIST Special Publication 800-132). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-132

National Institute of Standards and Technology. (2015). *Recommendation for key management, Part 1: General* (NIST Special Publication 800-57 Part 1 Revision 4). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-57pt1r4

OWASP Foundation. (n.d.). *Cryptographic storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

OWASP Foundation. (n.d.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Python Cryptographic Authority. (n.d.). *Symmetric encryption and authenticated encryption documentation*. cryptography. https://cryptography.io/en/latest/hazmat/primitives/aead/